

SatsPath — Docker Guide

Status: Ready for development and CI builds. Mainnet execution of payments is intentionally disabled by design.

Architecture

`docker-compose services`

<code>satspath-cli</code>	<code>ark-bridge</code> (<code>--profile</code>
<code>Rust / Debian Slim</code>	<code>bridge</code>) <code>Node 20 Slim</code>
<code>non-root uid:10001</code>	<code>non-root uid:10002</code>
<code>read-only rootfs</code>	<code>read-only rootfs</code>
<code>cap_drop: ALL</code>	<code>cap_drop: ALL</code>

<code>satspath-data</code>	Named volume: <code>.satspath/ registry</code>
(<code>Docker volume</code>)	survives container re-creation

Images

Image	Base	Size target	Binary
satspath-cli	debian:bookworm-slim	~25 MB	/usr/local/bin/satspath
satspath-ark-bridge	node:20-bookworm-slim	~120 MB	node dist/index.js

Both images use: - **Non-root user** (UID 10001 / 10002) - **Read-only root filesystem** (`read_only: true`)
- **All capabilities dropped** (`cap_drop: ALL`) - **no-new-privileges** security option - **OCI image labels**
for provenance

Quick Start

Prerequisites

- Docker 24 (or Podman 4) with BuildKit enabled
- `docker compose v2` plugin (or `docker-compose v1`)

1. Build all images

```
make build
# or manually:
docker build -t satspath-cli:latest .
docker build -t satspath-ark-bridge:latest -f ark-bridge/Dockerfile .
```

2. Initialize the registry

```
make init
# equivalent to:
docker compose run --rm satspath-cli init
```

This creates `.satspath/` inside the `satspath-data` named volume.

3. Register a profile

```
# With Lightning Address only:
docker compose run --rm satspath-cli register user@example.com \
  --lightning-address user@example.com

# With Arkade receive pointer (public-only, manual wallet):
docker compose run --rm satspath-cli register user@example.com \
  --arkade-uri "ark:ark1q..."

# With full Ark server + pubkey:
docker compose run --rm satspath-cli register user@example.com \
  --ark-server "https://ark.server.example" \
  --ark-pubkey "02..."
```

4. Get a routing quote

```
docker compose run --rm satspath-cli quote user@example.com 21000
```

5. Start the ARK bridge (optional)

The bridge is only needed for Ark swap validation and is disabled by default.

```
docker compose --profile bridge up -d ark-bridge
docker compose --profile bridge logs -f ark-bridge
```

Make targets

```
make help      # Show all targets
make build     # Build all images
make build-cli # Build CLI image only
make build-bridge # Build bridge image only
make run CMD="--help" # Run any CLI command
make shell     # Open a debug shell in the CLI container
make up        # Start bridge in background
make down      # Stop all services
make logs      # Tail all service logs
make scan      # Run Trivy vulnerability scan (requires trivy)
make clean     # Remove images and volumes
make smoke     # Build + verify --help / node --version
```

Security design

What is protected

Concern	Mitigation
Private keys in image	<code>.dockerignore</code> blocks <code>*.key</code> , <code>.satspath/</code> , <code>.env</code>
Root escalation	<code>no-new-privileges</code> , <code>cap_drop: ALL</code> , non-root users
Container escape	Read-only root filesystem + tmpfs for <code>/tmp</code> only
Dependency supply chain	<code>npm ci --ignore-scripts</code> (no postinstall scripts)
Secret injection	<code>.env</code> is gitignored; use Docker secrets or env at runtime
Vulnerability tracking	Trivy scan in CI via <code>.github/workflows/docker.yml</code>

What is intentionally not in Docker

- No wallet seed phrases
- No private spending keys
- No Arkade session tokens
- No mainnet payment execution

Layer caching strategy (Rust)

The Rust build uses `cargo-chef`:

```

Layer 1: cargo-chef planner → only re-runs when Cargo.toml/Cargo.lock change
Layer 2: cargo-chef cacher  → pre-builds all deps (very slow, cached)
Layer 3: builder            → compiles src/ (fast, re-runs on src change)
Layer 4: runtime            → copies single binary (~25 MB)

```

This means typical CI rebuilds take ~**30 seconds** instead of 10+ minutes.

Production checklist

- ☐ Push to a private registry (GHCR, ECR, etc.) — see `docker.yml` CI workflow
- ☐ Pin base image digests (replace `bookworm-slim` tags with `sha256:...`)
- ☐ Set `RUST_LOG` to `warn` in production
- ☐ Mount `satspath-data` volume to a backed-up external path
- ☐ Run `make scan` before each release to check for CVEs
- ☐ Review CI SARIF reports in GitHub Security tab

Troubleshooting

cargo: command not found in CI → The build runs inside the container; you don't need Cargo on the host.

Error: no such service: ark-bridge → Add `--profile bridge` flag: `docker compose --profile bridge up`.

Permission denied: /data → The `satspath-data` volume ownership may be wrong. Run:

```
docker compose run --rm --user root satspath-cli chown -R 10001:10001 /data
```

Build fails on `is_multiple_of` (pre-existing) → This is a known pre-existing issue in `satspath-router/src/lightning.rs` using a nightly-only Rust API. It does not affect the `satspath` CLI binary build, only `satspath-router` library checks on stable Rust. Unrelated to Docker.